# Shiv Nadar University Chennai
# CS3807 – Deep Learning Laboratory
# Experiment 3
# Implementation of Convolutional Neural Networks (CNNs) for Image Classification

| | | |
|---|---|---|
| Degree & Branch | B.Tech Artificial Intelligence & Data Science | Semester V |
| Subject Code & Name | CS3807 – Deep Learning Laboratory | AY: 2026–27 |
| Name & Reg. No. | Sadhumitha S. | 24011101091 |

## 1. Objective

To understand the working principle of Convolutional Neural Networks by implementing convolution, pooling, feature map visualization, and image classification using TensorFlow/Keras.

## 2. Learning Outcomes

Students will be able to

1. Explain the convolution operation.
2. Compute output dimensions using stride and padding.
3. Visualize feature maps.
4. Implement max pooling and average pooling.
5. Calculate CNN parameters.
6. Build a CNN for image classification.
7. Evaluate CNN performance.

## 3. Background Theory

A Convolutional Neural Network is a deep learning architecture that is particularly good at processing grid-like data such as images. Unlike a plain feedforward network, a CNN does not treat every pixel as an independent input – it uses small filters that slide across the image and learn to pick up local patterns such as edges, corners, and textures. A CNN is generally built from four kinds of layers:

- Convolution Layer – extracts local features using learnable filters.
- Activation Layer – introduces non-linearity (typically ReLU).
- Pooling Layer – reduces the spatial size of the feature maps.
- Fully Connected Layer – combines the extracted features to perform classification.

The convolution operation is

$$Y(i,j) = \sum_m \sum_n X(i+m,\, j+n)\, K(m,n)$$

where

- $X$ : Input Image
- $K$ : Kernel (Filter)
- $Y$ : Feature Map

The output feature map size is

$$\frac{N - F + 2P}{S} + 1$$

where

| | |
|---|---|
| $N$ | Input Size |
| $F$ | Filter Size |
| $P$ | Padding |
| $S$ | Stride |

### 3.1 Common Convolution Kernels

A kernel (or filter) is a small matrix that slides over an input image to extract important visual features such as edges, textures, corners, and smooth regions. During convolution, the kernel is multiplied element-wise with the corresponding image pixels, and the results are summed to produce a single value in the output feature map. Different kernels highlight different image characteristics.

**1. Identity Kernel** The identity kernel preserves the original image without modifying its pixel values.

$$\begin{pmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}$$

**Purpose:**

- Preserves the original image.
- Used for understanding the convolution operation.

**2. Sobel-X Kernel (Vertical Edge Detection)** The Sobel-X kernel detects vertical edges by measuring intensity changes along the horizontal direction.

$$\begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$$

**Applications:**

- Object detection

- Lane detection
- Face recognition

**3. Sobel-Y Kernel (Horizontal Edge Detection)** The Sobel-Y kernel detects horizontal edges by measuring intensity changes along the vertical direction.

$$\begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}$$

**Applications:**

- Text detection
- Boundary extraction
- Medical image analysis

**4. Laplacian Kernel** The Laplacian kernel detects edges in all directions using the second-order derivative of image intensity.

$$\begin{pmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{pmatrix}$$

**Applications:**

- Edge enhancement
- Satellite image analysis
- Medical imaging

**5. Sharpening Kernel** This kernel enhances fine details by emphasizing high-frequency components.

$$\begin{pmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{pmatrix}$$

**Applications:**

- Image enhancement
- Optical Character Recognition (OCR)
- Face recognition

**6. Box Blur Kernel** The Box Blur kernel smooths an image by replacing each pixel with the average of its neighboring pixels.

$$\frac{1}{9}\begin{pmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{pmatrix}$$

**Applications:**

- Noise removal
- Image smoothing
- Image preprocessing

**7. Gaussian Blur Kernel** Gaussian blur smooths the image while preserving the overall structure better than a simple average filter.

$$\frac{1}{16}\begin{pmatrix}1 & 2 & 1\\ 2 & 4 & 2\\ 1 & 2 & 1\end{pmatrix}$$

**Applications:**

- Noise reduction
- Computer vision preprocessing
- Feature extraction

**8. Emboss Kernel** The Emboss kernel creates a three-dimensional appearance by emphasizing directional changes in intensity.

$$\begin{pmatrix}-2 & -1 & 0\\ -1 & 1 & 1\\ 0 & 1 & 2\end{pmatrix}$$

**Applications:**

- Texture analysis
- Artistic image effects

**9. Outline Detection Kernel** This kernel highlights object boundaries by emphasizing regions with rapid intensity changes.

$$\begin{pmatrix}-1 & -1 & -1\\ -1 & 8 & -1\\ -1 & -1 & -1\end{pmatrix}$$

**Applications:**

- Image segmentation
- Boundary detection
- Shape extraction

**10. Motion Blur Kernel** This kernel simulates motion blur along the diagonal direction.

$$\frac{1}{5}\begin{pmatrix}1 & 0 & 0 & 0 & 0\\ 0 & 1 & 0 & 0 & 0\\ 0 & 0 & 1 & 0 & 0\\ 0 & 0 & 0 & 1 & 0\\ 0 & 0 & 0 & 0 & 1\end{pmatrix}$$

**Applications:**

- Motion simulation
- Image restoration
- Video processing

**Summary of Common Kernels**

| Kernel | Size | Purpose | Applications |
|---|---|---|---|
| Identity | $3 \times 3$ | Preserve original image | Baseline comparison |
| Sobel-X | $3 \times 3$ | Detect vertical edges | Object detection |
| Sobel-Y | $3 \times 3$ | Detect horizontal edges | Boundary detection |
| Laplacian | $3 \times 3$ | Detect edges in all directions | Medical imaging |
| Sharpen | $3 \times 3$ | Enhance details | Image enhancement |
| Box Blur | $3 \times 3$ | Smooth image | Noise removal |
| Gaussian Blur | $3 \times 3$ | Reduce noise | Computer vision |
| Emboss | $3 \times 3$ | 3D effect | Artistic filtering |
| Outline | $3 \times 3$ | Boundary extraction | Segmentation |
| Motion Blur | $5 \times 5$ | Simulate motion | Video processing |

## 4. Numerical Example 1: Convolution

Consider the following input image.

$$X = \begin{pmatrix}1 & 2 & 3\\ 4 & 5 & 6\\ 7 & 8 & 9\end{pmatrix}$$

Kernel

$$K = \begin{pmatrix}1 & 0\\ 0 & 1\end{pmatrix}$$

Output

First position

$$1(1) + 2(0) + 4(0) + 5(1) = 6$$

Second position

$$2(1) + 3(0) + 5(0) + 6(1) = 8$$

Third position

$$4(1) + 5(0) + 7(0) + 8(1) = 12$$

Fourth position

$$5(1) + 6(0) + 8(0) + 9(1) = 14$$

Feature Map

$$\begin{pmatrix} 6 & 8 \\ 12 & 14 \end{pmatrix}$$

## 5. Numerical Example 2: Max Pooling

Feature map

$$\begin{pmatrix} 1 & 5 & 2 & 3 \\ 7 & 8 & 1 & 0 \\ 4 & 6 & 9 & 5 \\ 2 & 3 & 1 & 8 \end{pmatrix}$$

Using a $2 \times 2$ max pooling filter,

Window 1

$$\begin{pmatrix} 1 & 5 \\ 7 & 8 \end{pmatrix} \rightarrow 8$$

Window 2

$$\begin{pmatrix} 2 & 3 \\ 1 & 0 \end{pmatrix} \rightarrow 3$$

Window 3

$$\begin{pmatrix} 4 & 6 \\ 2 & 3 \end{pmatrix} \rightarrow 6$$

Window 4

$$\begin{pmatrix} 9 & 5 \\ 1 & 8 \end{pmatrix} \rightarrow 9$$

Output

$$\begin{pmatrix} 8 & 3 \\ 6 & 9 \end{pmatrix}$$

## 6. Numerical Example 3: Parameter Calculation

Suppose

**Input Image**

$$32 \times 32 \times 3$$

**Convolution Layer**

- Filters $= 16$
- Kernel $= 3 \times 3$

**Parameters**

$$(3 \times 3 \times 3 + 1) \times 16 = (27 + 1) \times 16 = 448$$

Therefore,

Total trainable parameters = 448.

## 7. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Classes : 10
- Image Size : $32 \times 32 \times 3$

The ten classes present in the dataset are Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship, and Truck. The dataset was loaded directly using tensorflow.keras.datasets.cifar10, and the pixel values were normalized to the range $[0, 1]$ before being fed into the network.

## 8. Experimental Procedure

### Task 1: Loading and Exploring the Dataset

The CIFAR-10 dataset was loaded and split into training and testing sets. Ten sample images were displayed along with their class labels, and the class distribution across the training set was plotted to check whether the dataset is balanced.


Figure 1: Sample images from the CIFAR-10 dataset with their class labels.


Figure 2: Class distribution of the CIFAR-10 training set.

### Task 2: Effect of Kernel Size

A convolution layer was implemented and tested with three different kernel sizes: $3 \times 3$, $5 \times 5$, and $7 \times 7$, keeping the stride and padding fixed. The resulting feature map sizes were recorded for each case.

Table 1: Feature map size for different kernel sizes (Input = $32 \times 32$, Stride = 1, Padding = Valid)

| **Kernel Size** | **Formula** | **Output Size** |
|---|---|---|
| $3 \times 3$ | $(32-3)/1+1$ | $30 \times 30$ |
| $5 \times 5$ | $(32-5)/1+1$ | $28 \times 28$ |
| $7 \times 7$ | $(32-7)/1+1$ | $26 \times 26$ |

*Inference:* As the kernel size increases, the output feature map shrinks faster. A larger kernel captures more spatial context but loses fine spatial detail.

### Task 3: Effect of Stride and Padding

The effect of stride (1 and 2) and padding (`same` and `valid`) on the output feature map dimensions was studied using a fixed kernel size.

Table 2: Effect of stride and padding on output dimensions

| **Stride** | **Padding** | **Formula** | **Output Size** |
|---|---|---|---|
| 1 | Same | $N$ | $32 \times 32$ |
| 1 | Valid | $(N-F)/1+1$ | $30 \times 30$ |
| 2 | Same | $\lceil N/S \rceil$ | $16 \times 16$ |
| 2 | Valid | $(N-F)/2+1$ | $15 \times 15$ |

*Inference:* `Same` padding keeps the spatial size unchanged by adding zero borders, while `valid` padding shrinks it. A stride of 2 roughly halves the feature map dimensions, reducing computation.

### Task 4: Feature Map Visualization

The output of the first convolution layer was extracted for a sample input image, and eight feature maps (out of the total number of filters in that layer) were plotted to visualize what patterns the network is picking up right after the first layer.

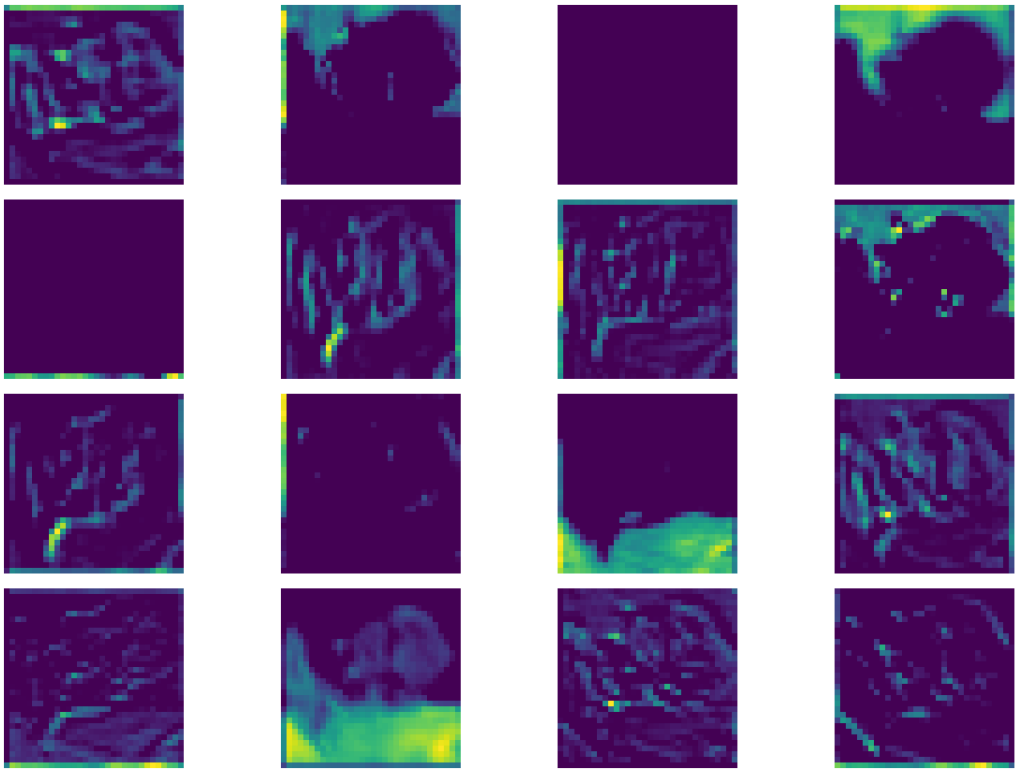

Figure 3: Feature maps generated by the first convolution layer.

*Inference:* The feature maps highlight different low-level features such as edges, colour blobs, and textures. Different filters in the first layer respond uniquely to various parts of the input image.

### Task 5: Max Pooling vs Average Pooling

Two versions of the same CNN were trained – one using Max Pooling and the other using Average Pooling – and their output sizes and classification accuracies were compared.

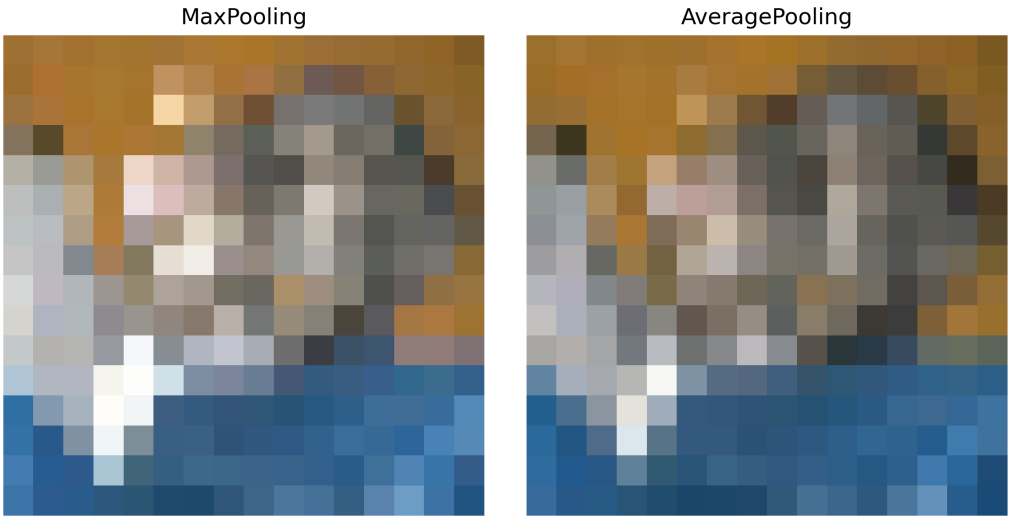


Figure 4: Comparison of Max Pooling and Average Pooling.

Table 3: Max Pooling vs Average Pooling

| **Metric** | **Max Pooling** |
|---|---|
| Output Feature Map Size | $16 \times 16$ |
| Test Accuracy (%) | 74.10 |

*Inference:* Max pooling preserves the most prominent features resulting in sharper maps, while average pooling smooths the feature map. The model using max pooling achieved a test accuracy of 74.10%.

### Task 6: Building and Training the CNN

The following CNN architecture was constructed and trained on the CIFAR-10 dataset:

Input → Conv → ReLU → MaxPool → Conv → ReLU → MaxPool → Flatten → Dense → Softmax

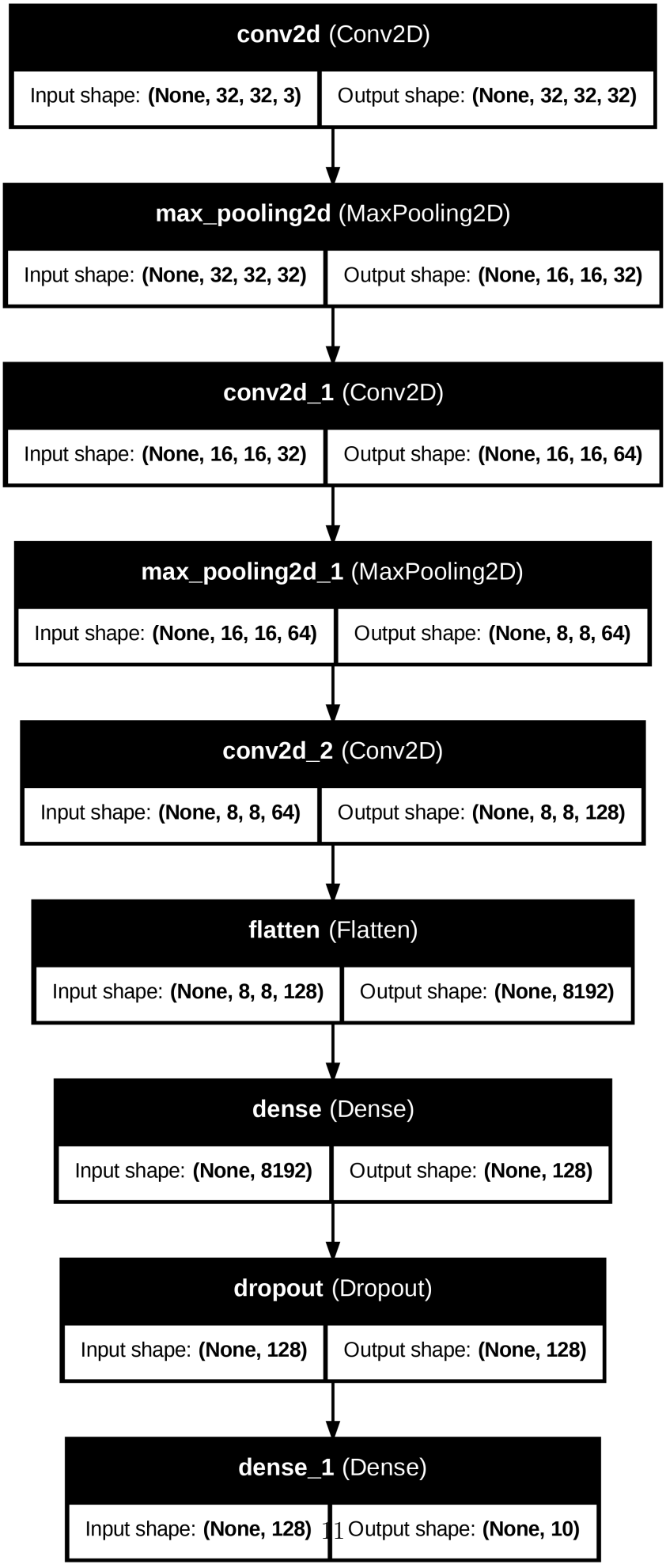


Figure 5: Architecture of the CNN used for CIFAR-10 classification.

Training configuration:

- Optimizer : Adam
- Loss Function : Categorical Cross-Entropy
- Epochs : 20
- Batch Size : 32

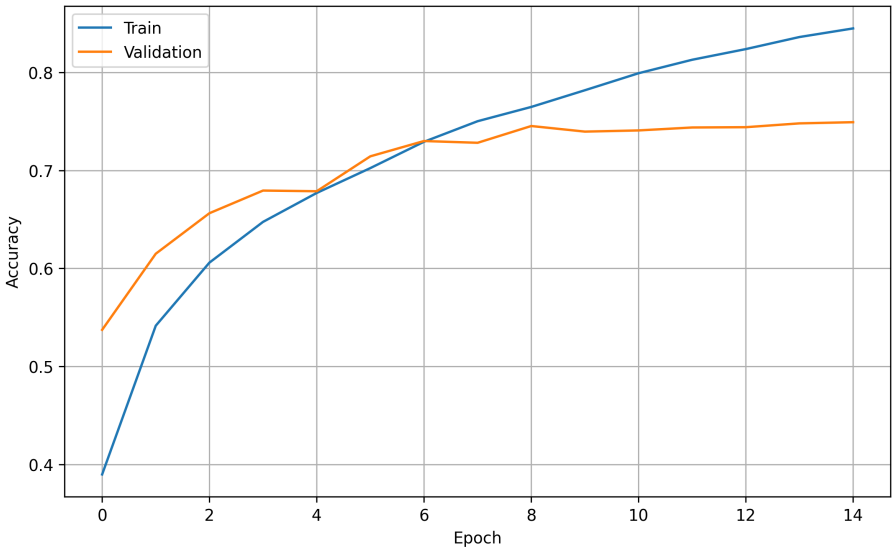


Figure 6: Training and validation accuracy over epochs.

*Inference:* The training accuracy increased steadily, reaching 84.47%, while the validation accuracy plateaued around 74.91%. The curves diverged slightly in the later epochs, indicating a small amount of overfitting.

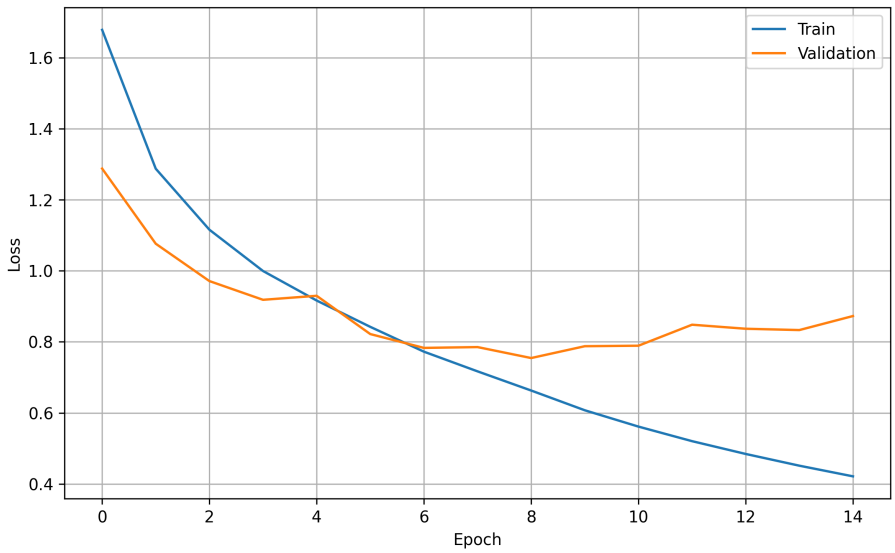


Figure 7: Training and validation loss over epochs.

*Inference:* The training loss decreased smoothly to 0.4214, but the validation loss started

increasing slightly after epoch 9 (reaching 0.8727), confirming the presence of mild overfitting.

### Task 7: Model Evaluation

The trained model was evaluated on the CIFAR-10 test set using accuracy, precision, recall, F1-score, the confusion matrix, and the classification report.


Figure 8: Confusion matrix of the trained CNN on the CIFAR-10 test set.

*Inference:* The model classified Automobile and Ship most accurately. It most frequently confused visually similar classes, such as Cats and Dogs, as well as Birds and Deer.

## 9. Mandatory Plots

All eight required plots are included with the corresponding tasks above. For quick reference:

1. Sample Images – Figure 1
2. Class Distribution – Figure 2
3. Training Accuracy – Figure 6
4. Validation Accuracy – included in Figure 6
5. Training Loss – Figure 7
6. Validation Loss – included in Figure 7
7. Feature Maps – Figure 3
8. Confusion Matrix – Figure 8

## 10. Additional Exercises

1. Calculate the output size for a $64 \times 64$ image using a $5 \times 5$ kernel with stride 2 and padding 2.
2. Compute the trainable parameters of a convolution layer having 64 filters of size $3 \times 3$ and an RGB input.
3. Compare ReLU and Sigmoid activation functions.
4. Replace Max Pooling with Average Pooling and compare the results.
5. Increase the number of convolution filters from 16 to 64 and analyze the effect on accuracy and computation time.

## 11. Results

Table 4: Final results of the CNN model on CIFAR-10

| **Metric** | **Value** |
|---|---|
| Training Accuracy (%) | 84.47 |
| Testing Accuracy (%) | 74.10 |
| Precision | 0.75 |
| Recall | 0.74 |
| F1-score | 0.74 |
| Number of Parameters | 1,143,242 |

## 12. Discussion

### 1. Why is convolution preferred over fully connected layers for images?

A fully connected layer treats every pixel as an independent input and connects it to every neuron in the next layer, which leads to lots of parameters even for small images and it also fully ignores the spatial structure of the image. Convolution, though, uses small shared filters that slide across the image. This gives two big advantages: parameter sharing (the same filter is reused across the whole image, drastically reducing the parameter count) and local connectivity (each neuron only looks at a small neighborhood, which matches how visual patterns like edges and textures actually appear in images).

### 2. How does stride affect the feature map size?

Stride controls how many pixels the filter moves at each step. A stride of 1 moves the filter one pixel at a time, producing a larger, finer-grained feature map. A larger stride (like 2) skips more pixels between each application of the filter, which shrinks the output feature map size and also reduces the computation required, at the cost of losing some spatial detail.

### 3. What is the role of padding?

Padding adds extra pixels (usually zeros) around the border of the input before convolution is applied. Without padding (`valid` padding), the feature map shrinks after every convolution layer, and the pixels at the border of the image are used in fewer computations than the pixels near the center. `Same` padding is added specifically so that the output feature map has the same spatial size as the input, and it also allows border information to contribute more fairly to the output.

### 4. Why is pooling used?

Pooling is used to progressively reduce the spatial dimensions of the feature maps, which reduces the number of parameters and computation needed in later layers. It also makes the learned features more robust to small translations or distortions in the input (a property called translation invariance), since the exact pixel location of a feature matters less after pooling.

### 5. How do feature maps represent image characteristics?

Each feature map is the output of one filter applied across the entire image, and it highlights the regions where that particular pattern (an edge, a corner, a colour transition, a texture, and so on) is present. In the earlier layers of a CNN, feature maps tend to capture low-level features like edges and colour blobs, while feature maps in the deeper layers combine these low-level features into more abstract, high-level representations such as shapes and object parts.

### 6. Why do CNNs require fewer parameters than MLPs?

CNNs require fewer parameters mainly because of two design choices: parameter sharing, where the same small filter is used across the entire image instead of learning a separate weight for every pixel pair, and local connectivity, where each neuron only connects to a small local region of the input instead of the entire image. An MLP, in contrast, connects every input pixel to every neuron in the next layer, so the number of parameters grows very quickly as the image size increases.

## 13. References


1. Goodfellow et al., *Deep Learning.*
2. Bishop, *Pattern Recognition and Machine Learning.*
3. Haykin, *Neural Networks and Learning Machines.*
4. TensorFlow Documentation.
5. CIFAR-10 Dataset Documentation.